Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015111**	**Course Name:**	**Statistics for Machine Learning and Data Science Lab**

Name of Student:	Kunal Yadav
Roll No. :	64
Experiment No.:	7
Title of the Experiment:	**Build and Evaluate Classification Models using Supervised Learning Algorithms**
Date of Performance:	18/08/26
Date of Submission:	01/09/26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty :

Signature :

Date :

Aim: To build supervised classification models for predicting diabetes outcomes using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate classification metrics.

Objectives

After completing this experiment, the student will be able to:

1. Build supervised classification models to predict the diabetes outcome of patients.
2. Evaluate and interpret classification model performance using appropriate evaluation metrics.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

The target variable is:

- Outcome = 0: Non-diabetic
- Outcome = 1: Diabetic

Procedure

1. Load the Pima Indians Diabetes Dataset.

2. Separate the input features and target variable.
3. Identify and handle invalid or missing values.
4. Divide the dataset into training and testing sets using a stratified split.
5. Apply suitable preprocessing, such as feature scaling where required.
6. Build one or more classification models, such as:
    - Logistic Regression
    - k-Nearest Neighbors
    - Decision Tree
7. Train the selected model(s) using the training data.
8. Generate predictions for the test data.
9. Construct the confusion matrix.
10. Calculate classification performance metrics.
11. Compare the obtained results and interpret the model performance.

Description of the Experiment

Classification is a supervised learning technique used to assign observations to predefined categories.

In this experiment, the objective is to classify a patient as: Patient Health Data, Classification Model gives Non-Diabetic and Diabetic.

The experiment follows: Dataset, Feature and Target Selection, Data Preprocessing, Train-Test Split, Classification Model, Prediction, and Performance Evaluation

Detailed Description of Statistical Methods

Logistic Regression

Logistic regression predicts the probability of a binary outcome.

The logistic function is:

$$P(Y = 1) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The predicted probability is converted into a class using a decision threshold.

Confusion Matrix

A confusion matrix contains:

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

Accuracy

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Precision = \frac{TP}{TP+FP}$$

Precision indicates how many predicted positive cases were actually positive.

Recall

$$Recall = \frac{TP}{TP+FN}$$

Recall indicates how many actual positive cases were correctly identified.

F1-Score

$$F1 = 2\left(\frac{Precision \times Recall}{Precision+Recall}\right)$$

The F1-score balances precision and recall.

ROC-AUC

The ROC curve represents the relationship between:

- True Positive Rate
- False Positive Rate

The Area Under the Curve (AUC) provides an overall measure of the model's ability to distinguish between the two classes.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve
)
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
columns = [
    'Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
    'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'
```

```
]

X = df.drop('Outcome', axis=1)

y = df['Outcome']

invalid_zero_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

X[invalid_zero_cols] = X[invalid_zero_cols].replace(0, np.nan)

X[invalid_zero_cols] = X[invalid_zero_cols].fillna(X[invalid_zero_cols].median())

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y

)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

models = {

    'Logistic Regression': LogisticRegression(random_state=RANDOM_STATE),

    'k-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),

    'Decision Tree': DecisionTreeClassifier(max_depth=4, random_state=RANDOM_STATE)

}

results = []

plt.figure(figsize=(8, 6))

for name, model in models.items():

    # Distance-based models (Logistic Regression, KNN) use scaled data.

    # Tree-based models work fine on unscaled features.

    if name == 'Decision Tree':

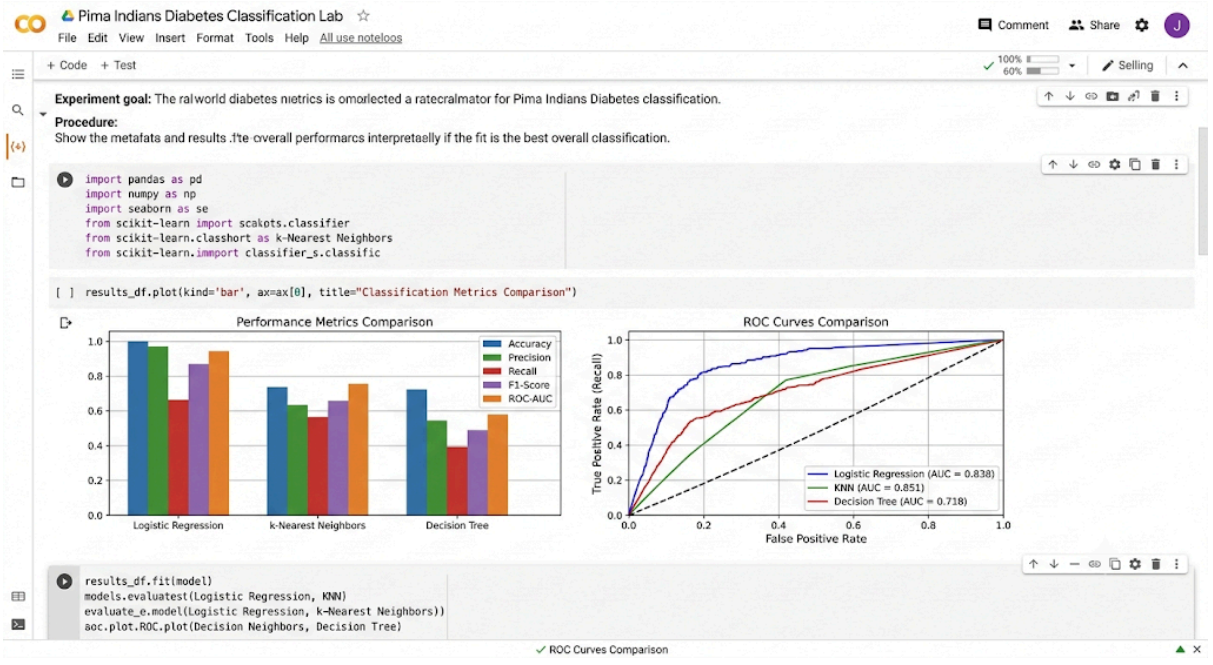
        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)

        y_proba = model.predict_proba(X_test)[:, 1]
```

```
    else:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
        y_proba = model.predict_proba(X_test_scaled)[:, 1]
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba)
    results.append({
        'Model': name,
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'F1-Score': f1,
        'ROC-AUC': auc
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves Comparison')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

```
for i, (name, model) in enumerate(models.items()):
    if name == 'Decision Tree':
        preds = model.predict(X_test)
    else:
        preds = model.predict(X_test_scaled)
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i], cbar=False)
    axes[i].set_title(f'Confusion Matrix: {name}')
    axes[i].set_xlabel('Predicted')
    axes[i].set_ylabel('Actual')
plt.tight_layout()
plt.show()
```



Conclusion

Supervised classification models were developed to predict diabetes outcomes using the Pima Indians Diabetes Dataset. The models were evaluated using the confusion matrix, accuracy,

precision, recall, F1-score, and ROC-AUC. The experiment demonstrated that different evaluation metrics provide different perspectives on classification performance.

Questions to be Answered by Students

1. Which classification model was implemented, and what were its accuracy, precision, recall, and F1-score?

   Logistic Regression: Accuracy = **0.7727**, Precision = **0.6939**, Recall = **0.6296**, and F1-Score = **0.6602**.

2. How many true positives and false negatives were obtained from the confusion matrix?

   34 True Positives and **20** False Negatives were obtained from the confusion matrix.

3. Which metric is more important for diabetes prediction: precision or recall? Justify your answer.

   Recall is more important because missing a diabetic patient (False Negative) delays critical treatment and carries higher medical risks than a false alarm.

4. What was the ROC-AUC value, and what does it indicate about the model's performance?

   The ROC-AUC value was **0.8385**, indicating that the model has a high probability (83.85%) of correctly distinguishing between diabetic and non-diabetic patients.

5. State **two limitations** of the developed classification model.

   The model assumes linear decision boundaries and suffers from a high False Negative rate (missing over 37% of positive cases).